REST vs GraphQL vs gRPC

how to choose an API style, and defend it

Watch the 3-min video

In one line: there is rarely a single winner: use REST for public and partner APIs, GraphQL as a backend-for-frontend when clients need flexible, aggregated views, and gRPC for fast, strongly typed calls between your own services. Most large systems run all three.

1 The real interview question

You have learned REST, GraphQL and gRPC. In an interview, the real question is: **which one would you pick here, and why?** "It depends" is only a good answer if you say what it depends on. Compare on six criteria: **performance and payload size**, **caching**, **browser and tooling support**, **coupling** between client and server, **streaming**, and, often forgotten, **your team's skills**.

	REST	GraphQL	gRPC
Performance & payload	●○○ JSON text	●●○ exact fields	●●● binary, HTTP/2
HTTP caching	●●● built in	●○○ POST: hard	●○○ custom
Browser & tooling	●●● everywhere	●●○ good clients	●○○ needs gRPC-Web
Loose coupling	●●● OpenAPI, loose	●●○ shared graph	●○○ shared .proto
Streaming	●○○ SSE / WS aside	●●○ subscriptions	●●● first-class
Team familiarity	●●● everyone knows	●●○ some	●○○ fewer

more dots = stronger on that criterion (a rule of thumb, not a benchmark)

Figure 1. A quick scorecard. No column wins every row, which is why big systems mix styles.

2 Strengths and costs of each style

REST: the most universal

Every language, tool, proxy and CDN understands it, and **HTTP caching works out of the box** (`Cache-Control`, `ETag`). Payloads are JSON text, so they are larger, and clients may need several calls for one screen. Contracts are usually documented with **OpenAPI**, and coupling stays loose: a client uses only the endpoints and fields it needs.

GraphQL: flexible data for many clients

It shines when many clients need different shapes of related data. One request fetches exactly what a screen needs. But HTTP caching is harder, the server must **guard against expensive queries** (depth and cost limits), and the schema becomes a **shared graph** that needs careful governance across teams.

gRPC: the fastest

Compact binary messages, HTTP/2 multiplexing, generated clients, **built-in deadlines** and first-class streaming. But browsers need gRPC-Web, payloads are not human-readable, and both sides must share the `.proto` files. So it fits best **where you control both ends**.

3 A common architecture uses all three

External partners call a **public REST API** through an API gateway. The mobile and web apps call a **GraphQL backend-for-frontend** (BFF) that aggregates data for each screen. Behind both, the **internal microservices talk over gRPC**.

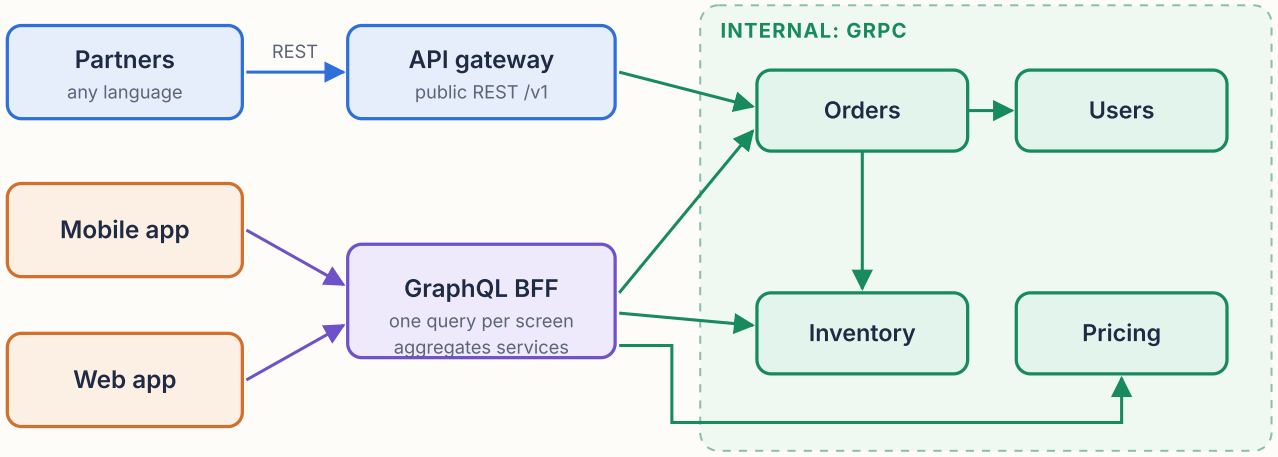


Figure 2. Each style sits where its strengths matter: REST at the public edge, GraphQL for first-party UIs, gRPC between services.

WORKED EXAMPLE: ONE PRODUCT PAGE, THREE STYLES

The mobile app sends one GraphQL query to the BFF over cellular (~100 ms round trip). The BFF fans out four gRPC calls inside the data centre (~1–3 ms each, in parallel), for product, price, stock and reviews, each with a 200 ms deadline. It assembles one JSON response shaped for the screen.

Meanwhile a partner's nightly job calls `GET /v1/products?updated_after=2026-11-01` on the public REST API, cached at the gateway for 60 s. Same data, three consumers, three styles, and each one is the simplest thing that works for its caller.

4 Choosing for a public partner API

Usually **REST**. Partners use every language and toolchain, and REST with JSON needs nothing special: **curl, Postman and any HTTP library just work**. It is cache-friendly, easy to document with OpenAPI, easy to version (`/v1`) and easy to secure with standard gateways (API keys, OAuth, rate limits).

- **GraphQL** can work for public APIs too (GitHub offers one), but you take on **query cost control** for clients you do not know.
- **gRPC** is a poor fit for unknown external clients: they need your `.proto` files, a gRPC toolchain, and cannot just try a call in a browser.

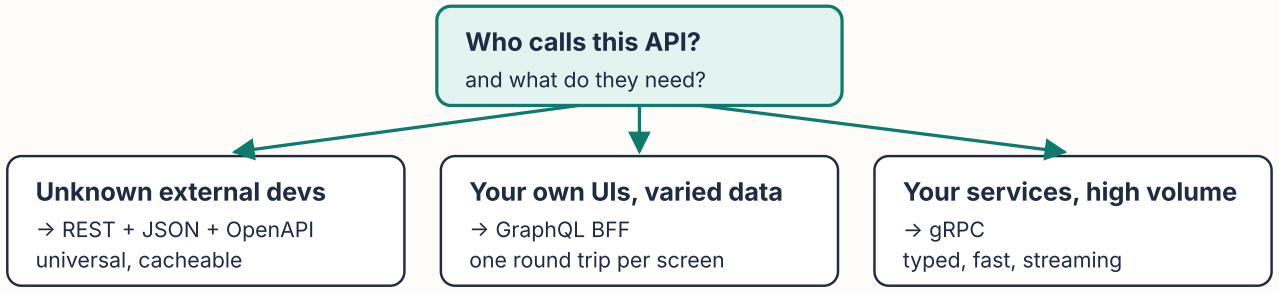


Figure 3. Start from the caller, not the technology.

COMMON MISTAKES

- Declaring one style “best” instead of matching it to the caller.
- Choosing gRPC for a public API, then building SDKs and proxies for every partner.
- Ignoring team skills: a style nobody on the team has run in production is a risk, not a feature.

5 Key terms

REST: resources + HTTP methods, usually JSON

BFF: backend-for-frontend: API per client type

GraphQL: client-shaped queries on a typed schema

OpenAPI: standard REST contract format

gRPC: binary RPC over HTTP/2 from .proto

API gateway: edge: auth, rate limits, routing

6 Interview questions

Q1. Which API style would you choose for a public partner API, and why? BASIC

REST. It is universal, works from any language with curl or any HTTP library, is cacheable, easy to document with OpenAPI, easy to version and easy to secure with standard gateways.

Q2. Compare REST, GraphQL and gRPC in one sentence each. BASIC

REST: universal and cacheable, but can over- or under-fetch. GraphQL: exact data per screen in one request, but harder caching and query-cost risk. gRPC: fastest and strongly typed with streaming, but needs shared .proto files and is awkward in browsers.

Q3. Design the API layer for a shop with partners, a mobile app and 30 microservices. SCENARIO

Public REST behind an API gateway for partners; a GraphQL BFF that serves each mobile or web screen in one round trip; gRPC with deadlines between the 30 services.

Q4. Two internal services exchange 20,000 small messages per second. Which style? SCENARIO

gRPC: binary payloads several times smaller than JSON, HTTP/2 multiplexing over a few connections, generated typed clients, deadlines, and streaming if needed.

Q5. Why not use GraphQL everywhere, including between services? TRICKY

Service-to-service calls have fixed shapes, so GraphQL's flexibility buys little, while you still pay for resolvers, query parsing and cost limits. gRPC gives stricter contracts and lower latency there.

Q6. When could GraphQL be right for a public API? TRICKY

When external developers build varied UIs over a rich, connected data model, as with GitHub's GraphQL API. You must then enforce cost-based rate limits and node limits, since you cannot trust unknown queries.

Next: SD-C-005 · API Resource Modeling & URI Design backendroadmap.com